

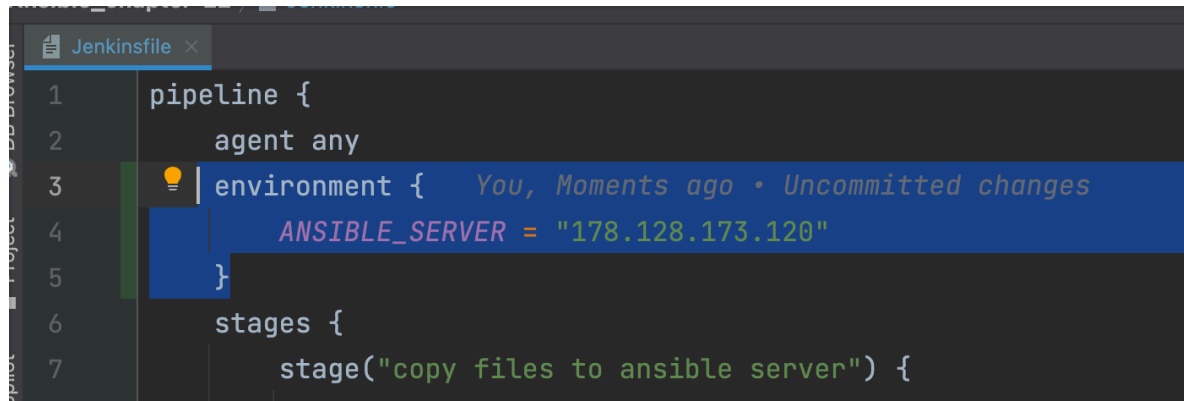
Module 15 - chapter 23

Project: Run Ansible from Jenkins Pipeline - Part 3

Optimizations

The remote IP address of ansible server is repeated several times, so we can improve that for maintainability.

So I create an environment var

A screenshot of a code editor window titled 'Jenkinsfile'. The code is a Jenkins pipeline definition. Line 3 shows an 'environment' block being defined. Line 4 shows the variable 'ANSIBLE_SERVER' being assigned the value '178.128.173.120'. Line 5 closes the 'environment' block. Line 6 shows a 'stages' block being defined. Line 7 shows a 'stage' named 'copy files to ansible server' being defined. A tooltip is visible over the 'environment' block, showing 'You, Moments ago • Uncommitted changes'.

```
1 pipeline {
2     agent any
3     environment {
4         ANSIBLE_SERVER = "178.128.173.120"
5     }
6     stages {
7         stage("copy files to ansible server") {
```

And use it where need it

A screenshot of a code editor window showing the continuation of the Jenkinsfile. Line 12 shows a shell step within the 'copy files to ansible server' stage. The command uses the 'ANSIBLE_SERVER' environment variable to specify the remote IP address. Line 15 shows a 'withCredentials' block that sets up an SSH key for the remote connection. Line 16 shows the 'scp' command being used to copy the key file to the remote server. Line 17 closes the 'withCredentials' block. Line 18 closes the 'script' block. Line 19 closes the 'stage' block. Line 20 closes the 'stages' block.

```
12     sh "scp -o StrictHostKeyChecking=no ansible/* root@${ANSIBLE_SERVER}:178.128.173.120:/root"
13
14     withCredentials([sshUserPrivateKey(credentialsId: 'ec2-server-key', keyFileVariable: 'keyFile',
15     sh 'scp $keyFile root@178.128.173.120:/root/ssh-key.pem'
16     })
17 }
18 }
19 }
20 }
```

Another thing that we can improve is to automate the manual process of configuring the ansible server using a script
-> sshScript

When we created the ansible droplet we manually ssh into it and installed ansible and install python3-boto3 (see Module 15 - chapter 21)

So now we can create a shell file with that configuration

```
Jenkinsfile x prepare-ansible-server.sh x
1 1 ▶ #!/user/bin/env bash
2
3 apt update
4 apt install ansible -y
5 apt install python3-boto3
6 | You, Moments ago • Uncommitted change
```

And then we can use that in jenkinsfile 💡 this way if we have ansible servers that get destroyed or created dynamically we can configure them efficiently.

```
21 stage ("execute ansible playbook") {
22     steps {
23         script {
24             echo "calling ansible playbook to configure EC2 instances"
25             def remote = [:]
26             remote.name = "ansible-server"
27             remote.host = env.ANSIBLE_SERVER
28             remote.allowAnyHosts = true
29
30             withCredentials([sshUserPrivateKey(credentialsId: 'ansible-server-key', keyFileVariable: 'keyFile',
31             remote.user = user
32             remote.identityFile = keyFile
33             sshScript remote, script: "prepare-ansible-server.sh" You, 5 minutes ago • Uncommitted
34             sshCommand remote: remote, command: "ansible-playbook my-playbook.yaml"
35         }
36     }
37 }
38 }
```

The script will executed in the pipeline every time the build runs.

✅ ansible-pipeline

Stage View



```
calling ansible playbook to configure EC2 instances
[Pipeline] withCredentials
Masking supported pattern matches of $keyFile
[Pipeline] {
[Pipeline] sshScript
Executing script on ansible-server[178.128.173.120]: /var/jenkins_home/workspace/ansible-pipeline/prepare-ansible-server.sh

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://mirrors.digitalocean.com/ubuntu noble InRelease
Get:3 http://mirrors.digitalocean.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 https://repos-droplet.digitalocean.com/apt/droplet-agent main InRelease
Hit:5 http://mirrors.digitalocean.com/ubuntu noble-backports InRelease
Get:6 http://mirrors.digitalocean.com/ubuntu noble-updates/main amd64 Packages [1191 kB]
Get:7 http://mirrors.digitalocean.com/ubuntu noble-updates/universe amd64 Packages [1683 kB]
Fetched 3000 kB in 11s (274 kB/s)
Reading package lists...
```

This step is **optional**, it depends on how we handle our servers

```
29
30         withCredentials([sshUserPrivateKey(credentialsId: 'ansible-server-key', keyFileVariable:
31             remote.user = user
32             remote.identityFile = keyFile
33         //             sshScript remote: remote, script: "prepare-ansible-server.sh" You, 34 minutes
34             sshCommand remote: remote, command: "ansible-playbook my-playbook.yaml"
35         }
36     }
37 }
38 }
```

Another optimisation but we won't implement in this chapter:
We could also automate creating the AWS credentials (~/.aws/credentials) on the remote ansible server by creating the credential in jenkins

	jenkins_aws_secret_access_key	jenkins_aws_secret_access_key	Secret text	
	jenkins_aws_access_key_id	jenkins_aws_access_key_id	Secret text	

- AWS access key
- AWS secret access key

If we need to access AWS API we need AWS credentials like we did in Module 15 - chapter 20. Locally the credentials get stored in -> ~/.aws/credentials

Module 12 Q&A
Yesterday Q:

Module 14 - chapter 9
Yesterday Backup EC2 Volumes: Automate creating...

Module 15 - chapter 20
Yesterday Deploying Application in K8s

Module 10 - chapter 7
Yesterday Complete Demo Project - Deploying Appli...

Previous 7 Days

Module 15 - chapter 19
Tuesday Dynamic Inventory for EC2 Servers

Module 14 - chapter 3
Tuesday Getting familiar with Boto

Module 15 - chapter 18
Monday Project: Terraform & Ansible

.gitignore and untracked files when branching

→ finds EKS API server URL + exec-based auth
→ shells out to `aws eks get-token` using your local AWS credentials/IAM role
→ gets a bearer token, calls the EKS API server

One practical implication for your play: whatever machine actually runs this `Ansible` play (your laptop, or a CI runner, or wherever) needs:

- a valid `kubeconfig` pointing at the EKS cluster (or `KUBECONFIG` env var set)
- AWS credentials available (env vars `~/.aws/credentials` or an instance/task IAM role) that map to an IAM identity with entry in the cluster's `aws-auth` ConfigMap / EKS access entries

If you're running this from CI (Jenkins, per your other project) rather than a laptop, that's usually the part that needs the care — making sure the CI agent has both the `kubeconfig` and the right IAM role, not a human's local config.

— end tangent —

WHICH cluster to connect to?

HOW to connect to cluster?

call the AWS API — e.g. to create/modify EC2 instances, security groups, S3 buckets, etc. via Ansible's `amazon.aws` collection modules. Those modules need `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` available on the machine running them.

instead of manually SSH-ing into the ansible-server yourself and hand-typing your AWS access key/secret into a `~/.aws/credentials` file (or exporting them as env vars) so Ansible can talk to AWS — automate that the same way you automated the pem file: store the two values as Jenkins credentials (which you've already done — good), then have the Jenkinsfile pull them via `withCredentials` and push them onto the ansible-server itself (e.g. write `~/.aws/credentials`, or export them before running `ansible-playbook`).